IPC Mechanism Selection and Justification

1. Need for Inter-Process Communication

In Week 4, the STC89C52 Educational Microcontroller Simulator was redesigned from a standalone simulator into a multi-process system consisting of three independent processes:

- **UI Process – handles user interaction and displays simulator information.**
- **Core Process – performs CPU execution and manages Memory, Stack and FIFO Queue operations.**
- **Logging Process – receives execution and system messages and stores them in the log file.**

Since these components execute as separate processes, they cannot directly access each other's local memory. Therefore, an Inter-Process Communication (IPC) mechanism is required to exchange commands, responses and log messages.

2. Selected IPC Mechanism

The IPC mechanism selected for the simulator is TCP Socket Communication.

Java socket programming using `ServerSocket` and `Socket` is used to establish communication between the independent simulator processes.

The communication architecture uses two main ports:

Port	Communication	Purpose
5001	UI Process ↔ Core Process	Sends simulator commands and receives responses/state information
5002	UI/Core → Logging Process	Sends execution and system log messages to the Logger

This creates a clear communication interface between the three processes while allowing each process to execute independently.

3. Communication Flow

When the user performs an operation such as LOAD, STEP or RESET, the UI Process sends the corresponding command to the Core Process through the TCP socket connection on port 5001.

The Core Process receives the command and performs the required CPU or simulator operation. The resulting state or response is then returned to the UI Process.

Execution-related messages generated by the simulator are sent to the Logging Process through port 5002. The Logging Process receives these messages and records them in `simulator.log`.

The basic communication flow is:

UI Process → TCP Socket → Core Process → CPU / Memory / Stack / FIFO Queue

and

UI/Core → TCP Socket → Logging Process → simulator.log

4. Why TCP Sockets Were Selected

TCP sockets were selected for the following reasons:

Reliable Communication:
TCP provides reliable and ordered delivery of data. This is important because simulator commands such as LOAD, STEP and RESET must reach the Core correctly and in the expected order.

Independent Process Communication:
Sockets allow the UI, Core and Logging components to run as independent processes while still exchanging information.

Bidirectional Communication:
The UI must send commands to the Core and receive responses from it. TCP sockets support this two-way communication.

Clear Process Interfaces:
Using separate ports provides a clear interface between the processes. Port 5001 is used for Core communication, while port 5002 is used for logging communication.

Java Support:
Java provides built-in networking classes such as `Socket` and `ServerSocket`, making socket-based IPC suitable for the Java implementation of the simulator.

Platform Independence:
Socket communication is supported across different operating systems. This makes the communication architecture less dependent on a particular operating system.

Future Extensibility:
The socket-based design can be extended in the future so that simulator components can communicate even when they are executed on different machines.

5. Use of LinkedBlockingQueue

`LinkedBlockingQueue` is also used in the multi-process simulator, but it serves a different purpose.

It is used inside individual processes for thread-safe communication between threads. For example, the Core Process can safely transfer received commands to its execution worker through a blocking queue.

Therefore:

TCP Socket → Inter-Process Communication

LinkedBlockingQueue → Intra-Process / Inter-Thread Communication

This distinction is important because the blocking queue itself does not provide communication between the three independent JVM processes.

6. Final Design Decision

TCP socket communication was selected because it provides a reliable, structured and platform-independent communication mechanism for the multi-process simulator. It allows the UI, Core and Logging processes to remain separate while maintaining controlled communication between them.

The combination of TCP sockets for process-to-process communication and LinkedBlockingQueue for thread-to-thread communication within a process provides a clear separation between IPC and internal concurrency mechanisms.

This design supports the main objective of Week 4: transforming the original standalone simulator into a modular multi-process software system with clearly defined communication between its components.